

About Dataset Context "Predict behavior to retain customers. You can analyze all relevant customer data and develop focused customer retention programs." [IBM Sample Data Sets]

Content Each row represents a customer, each column contains customer's attributes described on the column Metadata.

The data set includes information about: • Customers who left within the last month – the column is called Churn

- Services that each customer has signed up for – phone, multiple lines, internet, online security, online backup, device protection, tech support, and streaming TV and movies
- Customer account information – how long they've been a customer, contract, payment method, paperless billing, monthly charges, and total charges
- Demographic info about customers – gender, age range, and if they have partners and dependents

The project will cover: ☒ Data Loading & Preprocessing

☒ Handling Missing Values & Data Cleaning

☒ Descriptive Statistics & Summary

☒ Visualizations (Univariate, Bivariate, Multivariate Analysis)

☒ Insights on Churn Behavior

```
#Load a important libraries for EDA project
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Load the data into df variable
```

```
df = pd.read_csv("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

```
#Lets check data into df variable
```

```
df
```

	er	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multip.
	ale	0	Yes	No	1	No	No phon
	ale	0	No	No	34	Yes	
	ale	0	No	No	2	Yes	
	ale	0	No	No	45	No	No phon
	ale	0	No	No	2	Yes	
	...	...	...	...	...	...	
	ale	0	Yes	Yes	24	Yes	
	ale	0	Yes	Yes	72	Yes	
	ale	0	Yes	Yes	11	No	No phon
	ale	1	Yes	No	4	Yes	
	ale	0	No	No	66	Yes	

```
# As we seen data is huge . so all columns not showing aafter print df
# so lets check the total columns name
```

```
df.columns
```

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
      'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
      'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

```
#Lets check the shape of the data
```

```
print("Dataset shape : -",df.shape)
```

```
Dataset shape : - (7043, 21)
```

1 - The dataset cotains 7043 rows customers and 21 col;uimns (feature) 2 - each row represent a unique customer with various attribute related to demographic , service and biling 3- The dataset is sufficient for meaningful analysis amnd predictive modeling 4 - with 21 features , we have a good mix of categorical and numerical data to explore the customer behaviors

```
# so lets check data types of each columns
```

```
print(df.dtypes)
```

```
customerID      object
gender          object
SeniorCitizen   int64
Partner         object
Dependents      object
tenure          int64
PhoneService    object
MultipleLines   object
InternetService object
OnlineSecurity  object
OnlineBackup    object
DeviceProtection object
TechSupport     object
StreamingTV     object
StreamingMovies object
Contract        object
PaperlessBilling object
PaymentMethod   object
MonthlyCharges  float64
TotalCharges    object
Churn           object
dtype: object
```

✓ insights

1 - Most of the columns are categorical(object types) 2 - Senior citizen is stored as int 64. but since it's a binary variable (0,1) so it can be treated as category 3 - Monthly charges are correctly stored as float, good for analysis 4 - Total charges is stored as object, which seems incorrect since it represents numerical values. It may contain missing values so it can be represented as an object we need to convert it to a float for proper analysis

```
# convert total charges to numeric
```

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

```
# Let's check the conversion
```

```
df["TotalCharges"].dtypes
```

```
dtype('float64')
```

```
# check the missing values in each column
```

```
missing_values = df.isnull().sum()
```

```
missing_values
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	11
Churn	0

dtype: int64

missing_values[missing_values >0]

	0
TotalCharges	11

dtype: int64

✓ insights

1 - Only one columns total charges has missing values (11 missing entries) 2 - Since totalcharges is now numerical, this missing values were likely empty shapes in original dataset 3 - possible way top handle missing values

- fill them with mean or median

- fill them with zero ,but total charges is possibly not be zero -drtop these 11 rows, but if we do 11 customers also drop.from 7043
- so better so filling missing values
- fill with median meadian is good if we may contain outlier

```
df["TotalCharges"].median()
```

```
1397.475
```

```
# so lets replace missing values
```

```
df["TotalCharges"].fillna(df["TotalCharges"].median(),inplace = True)
```

/tmp/ipykernel_661/3438480645.py:1: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col

```
df["TotalCharges"].fillna(df["TotalCharges"].median(),inplace = True)
```

```
df["TotalCharges"].isnull().sum()
```

```
np.int64(0)
```

```
# Lets check for the duplicate rcords
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
# since the no .of duplicate record is zero  
# it means each row contains the unique customer data
```

```
df.columns
```

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',  
      'tenure', 'PhoneService', 'MultipleLines', 'InternetService',  
      'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',  
      'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',  
      'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],  
      dtype='object')
```

```
categorical_cols =[ 'gender', 'SeniorCitizen', 'Partner', 'Dependents',  
                   'PhoneService', 'MultipleLines', 'InternetService',  
                   'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',  
                   'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',  
                   'PaymentMethod', 'Churn'],
```

```
# lets check the unique valoues in each category
```

```
df["gender"].unique()
```

```
array(['Female', 'Male'], dtype=object)
```

```
for col_name in categorical_cols[0]:  
    print(f"unique value in {col_name} is :-")  
    print(df[col_name].unique(),"\n")
```

```
unique value in gender is :-  
['Female' 'Male']
```

```
unique value in SeniorCitizen is :-  
[0 1]
```

```
unique value in Partner is :-  
['Yes' 'No']
```

```
unique value in Dependents is :-  
['No' 'Yes']
```

```
unique value in PhoneService is :-  
['No' 'Yes']
```

```
unique value in MultipleLines is :-  
['No phone service' 'No' 'Yes']
```

```
unique value in InternetService is :-  
['DSL' 'Fiber optic' 'No']
```

```
unique value in OnlineSecurity is :-  
['No' 'Yes' 'No internet service']
```

```
unique value in OnlineBackup is :-  
['Yes' 'No' 'No internet service']
```

```
unique value in DeviceProtection is :-  
['No' 'Yes' 'No internet service']
```

```
unique value in TechSupport is :-  
['No' 'Yes' 'No internet service']
```

```
unique value in StreamingTV is :-  
['No' 'Yes' 'No internet service']
```

```
unique value in StreamingMovies is :-  
['No' 'Yes' 'No internet service']
```

```
unique value in Contract is :-  
['Month-to-month' 'One year' 'Two year']
```

```
unique value in PaperlessBilling is :-  
['Yes' 'No']
```

```
unique value in PaymentMethod is :-  
['Electronic check' 'Mailed check' 'Bank transfer (automatic)'  
 'Credit card (automatic)']
```

```
unique value in Churn is :-  
['No' 'Yes']
```

```
# lets check for the numerical faetures  
# also check for the outlier
```

Start coding or generate with AI.